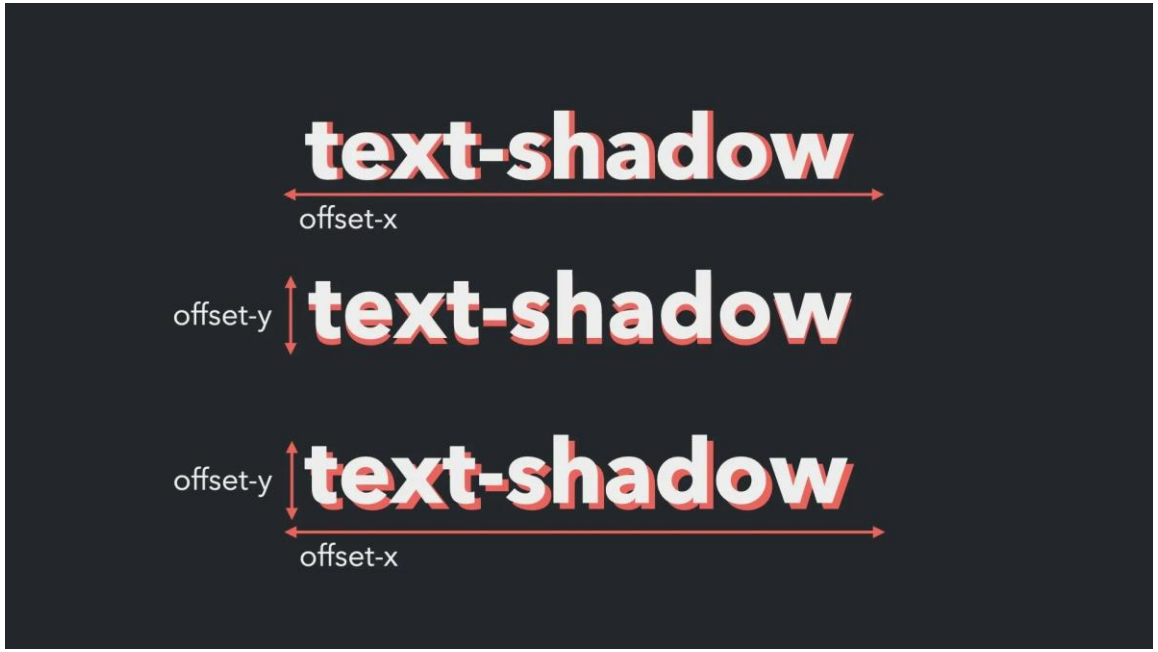
# Эффекты и тени, адаптивная вёрстка

Докладчики: Корнев Андрей, Голубков Кирилл, Шипелин Алексей

# CSS-свойство **text-shadow**

- CSS-свойство **text-shadow** добавляет тени к тексту. Свойство задаётся разделённым запятыми списком теней, которые будут применены к тексту и к любым его свойствам decorations. Любая тень описывается комбинацией смещений по осям X и Y относительно элемента, радиусом размытия и цветом.Это свойство определено как разделённый запятыми список теней.



# CSS-свойство **text-shadow**

- Каждая тень определена как два или три значения <длина>, за которыми следует необязательное значение <цвет>.
- Первые два значения определяют параметры <смещение-x> и <смещение-y>.
- Значение <цвет> определяет цвет тени.

```
.text2 {
    text-shadow: 5px 10px;
}
```

**Far out in the uncharted backwaters of the unfashionable end of the western spiral arm of the Galaxy...**

# CSS-свойство **text-shadow**

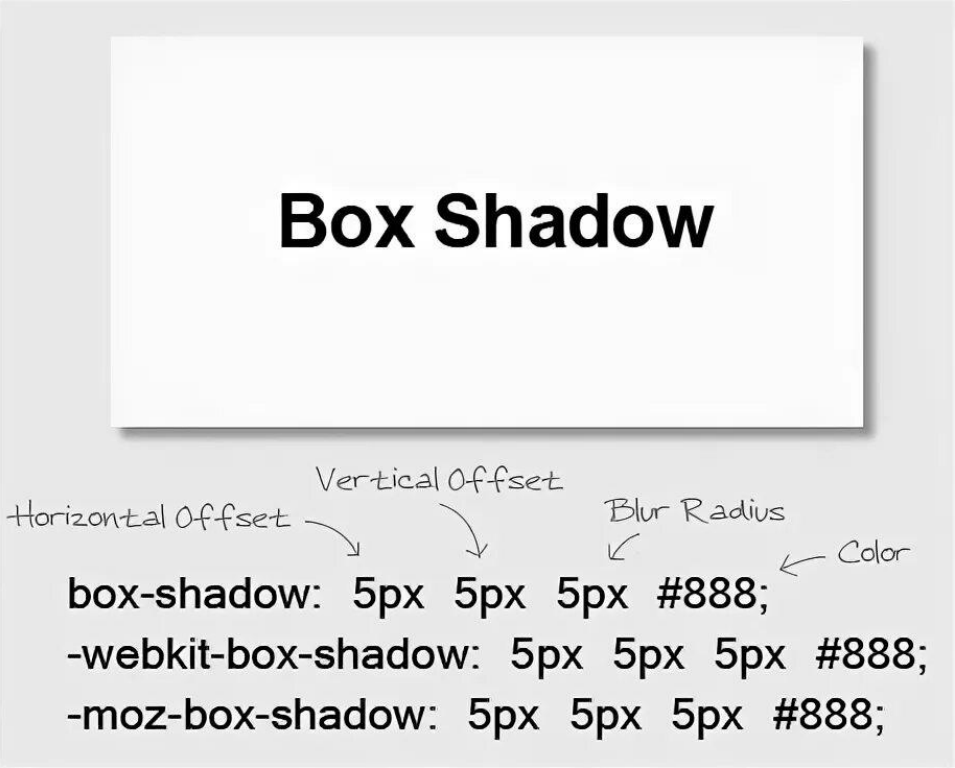
- Третье значение задаёт <радиус-размытия>.
- Если определены больше чем одна тень, тени накладываются друг на друга в порядке обратном порядку их определения, что значит, первая тень окажется поверх последующих.

```
.text3 {
    text-shadow:
        1px 1px 2px red,
        0 0 16px blue,
        0 0 3px blue;
}
```

**Far out in the uncharted backwaters of the unfashionable end of the western spiral arm of the Galaxy...**

# CSS-свойство box-shadow

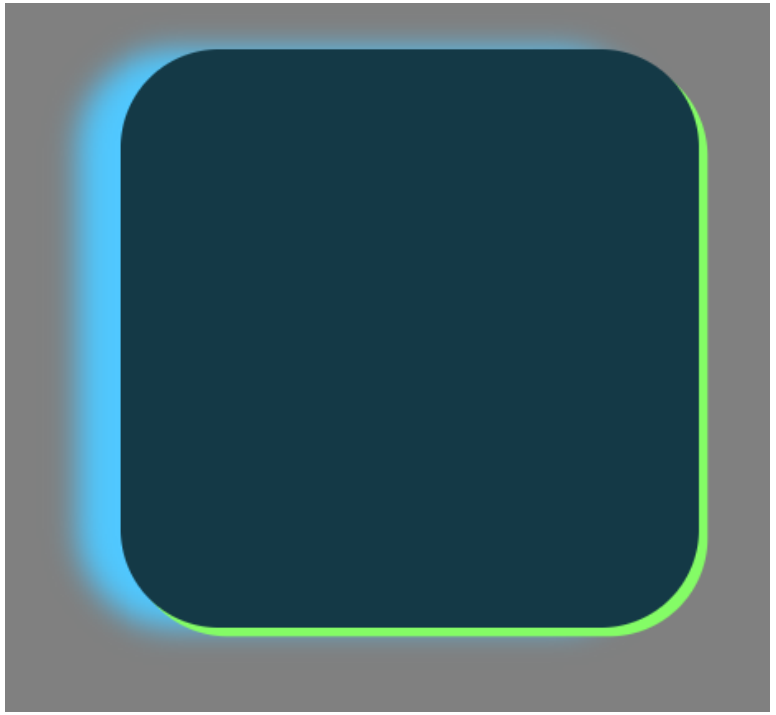
- CSS-свойство **box-shadow** добавляет тень к элементу. Через запятую можно задать несколько теней. Тень описывается смещениями по оси X и Y относительно элемента, радиусом размытия и распространения, а также цветом.



# CSS-свойство box-shadow

- Свойство box-shadow включает отбрасывание тени от границ практически любого элемента. Если для элемента с тенью задан border-radius, то отбрасываемая тень также будет с закруглёнными углами.

```
.box5 {
    width: 200px;
    height: 200px;
    /*Отбрасываемая тень также будет с закруглёнными углами*/
    background-color: rgb(20, 57, 70);
    border-radius: 34px;
    /* Любое количество теней через запятую */
    box-shadow:
        3px 3px rgb(132, 252, 102),
        -15px 0 10px rgb(80, 200, 255);
}
```



# CSS-свойство box-shadow

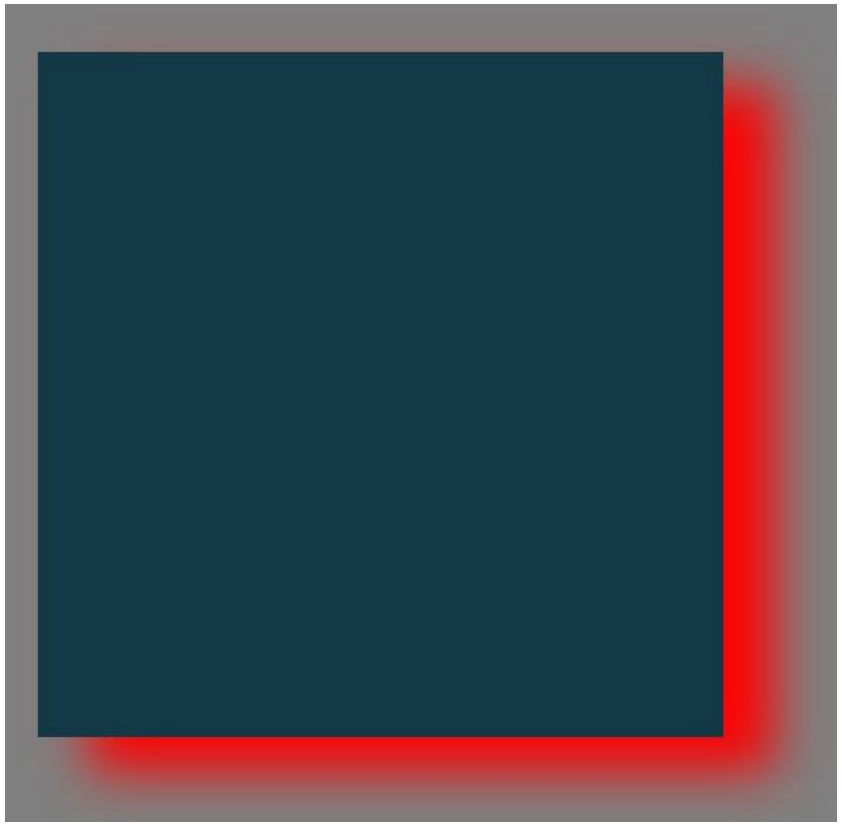
- Каждая тень определяется следующим образом:
- Два, три или четыре значения длины (<length>).
- Если указаны только два значения, они будут означать смещение по X (<offset-x>) и смещение по Y (<offset-y>). Необязательное значение цвета (<color>).

```
.box1 {
    width: 200px;
    height: 200px;
    background-color: rgb(20, 57, 70);
    /* offset-x | offset-y  color */
    box-shadow: 20px -16px teal;
}
```

# CSS-свойство box-shadow

- Третье значение устанавливает радиус размытия (<blur-radius>).

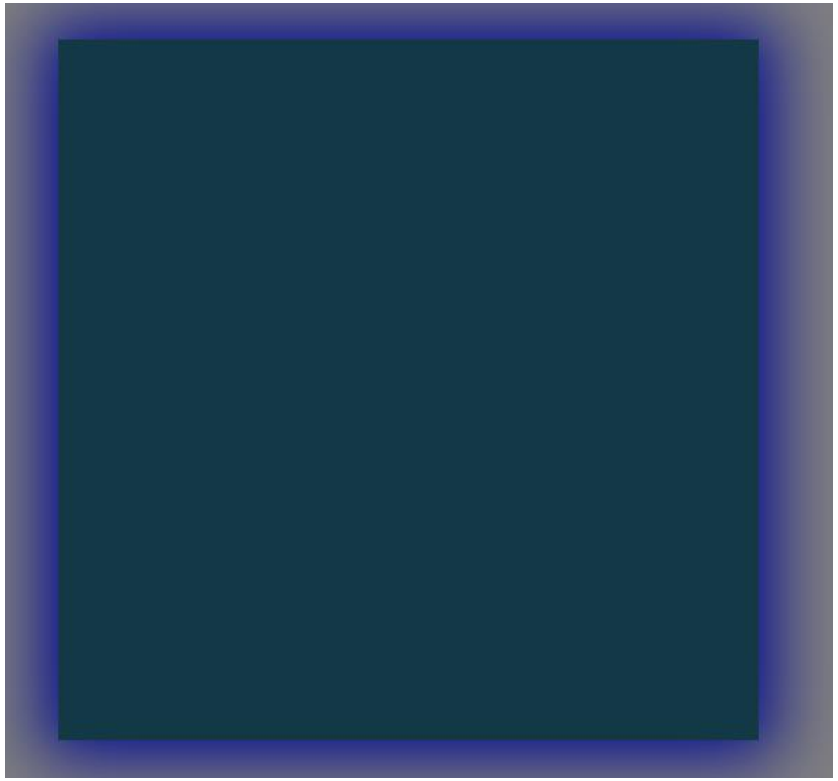
```
.box2 {
    width: 200px;
    height: 200px;
    background-color: rgb(20, 57, 70);
    /* offset-x | offset-y | blur-radius | color */
    box-shadow: 15px 10px 15px red;
}
```



# CSS-свойство box-shadow

- Четвёртое значение задаёт радиус распространения (<spread-radius>).

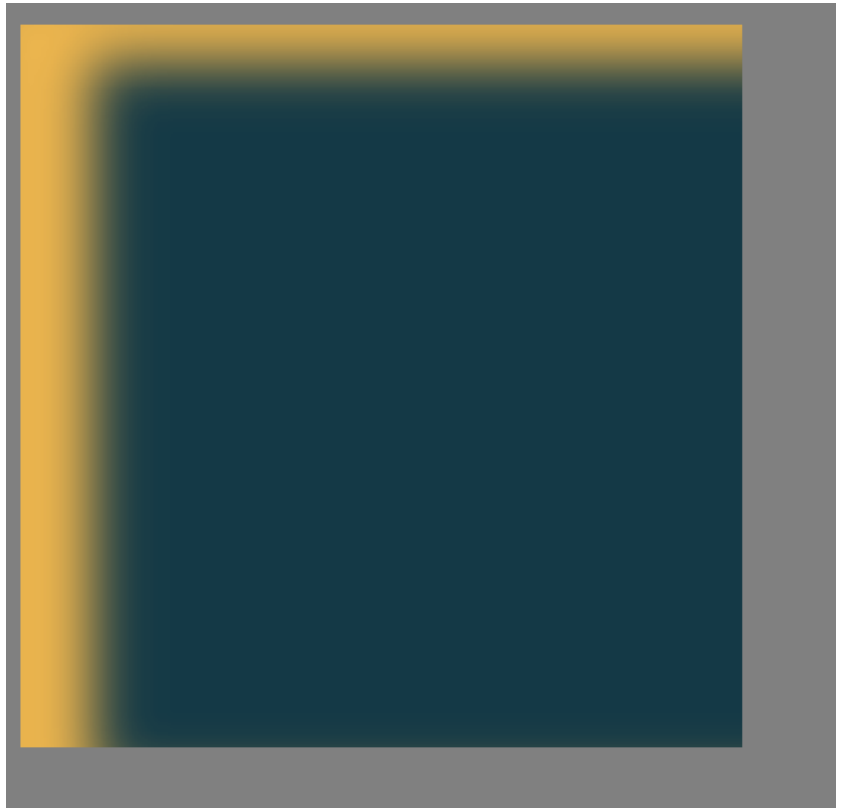
```
.box3 {
    width: 200px;
    height: 200px;
    background-color: rgb(20, 57, 70);
    /* offset-x | offset-y | blur-radius | spread-radius | color */
    box-shadow: 0px 0px 20px 5px rgb(2, 9, 143);
}
```



# CSS-свойство box-shadow

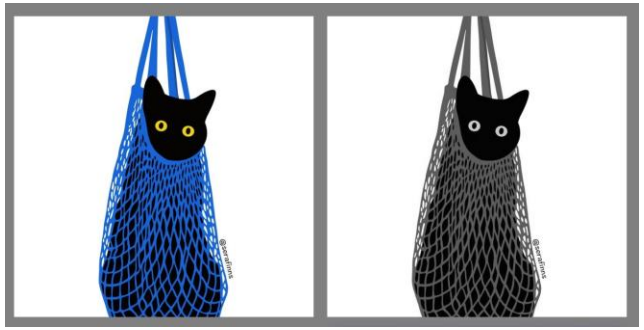
- Необязательное ключевое слово inset.

```
.box4 {
    width: 200px;
    height: 200px;
    background-color: rgb(20, 57, 70);
    /* inset | offset-x | offset-y | color */
    box-shadow: inset 20px 10px 15px rgb(238, 182, 78);
}
```



**grayscale() — делает элемент чёрно-белым**

```
.photo_grayscale {
    filter: grayscale(100%);

}
```



**blur() — размывает элемент**

```
.photo_blur {
    filter: blur(5px);
}
```

**brightness() — изменяет яркость элемента**

```
.photo_brightness {
    filter: brightness(0.4);
}
```

**contrast() — изменяет контраст элемента**

```
.photo_contrast {
    filter: contrast(200%);
}
```

**drop-shadow() — добавляет тень по форме элемента**

```
.photo_drop-shadow {
    filter: drop-shadow(16px 16px 20px rgb(131, 131, 240));
}
```

**hue-rotate() — изменяет оттенок (сдвигает цвета по цветовому кругу)**

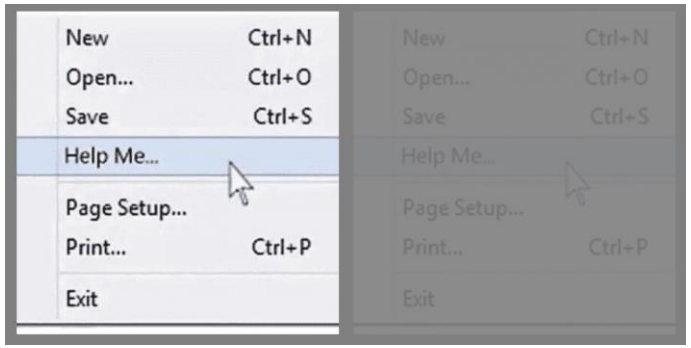
```
.photo_hue-rotate {
    filter: hue-rotate(90deg);
}
```

**invert() — инвертирует цвета элемента**

```
.photo_invert {
    filter: invert(100%);
}
```

**opacity() — изменяет прозрачность элемента**

```
.photo_opacity {
    filter: opacity(10%);
}
```

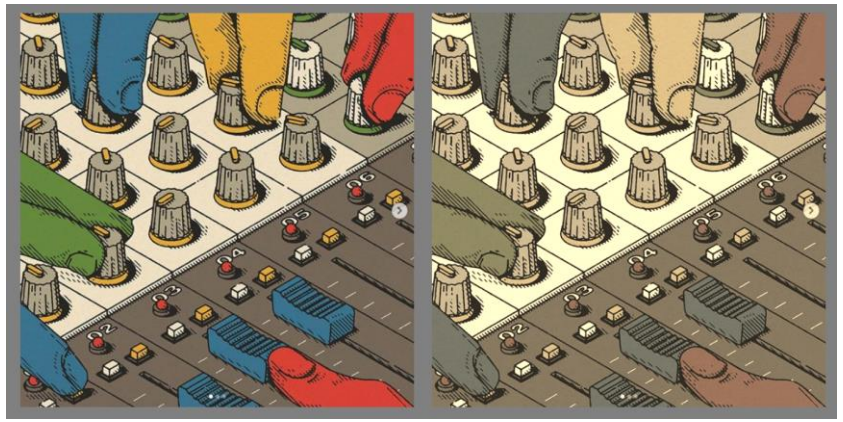


**saturate() — изменяет насыщенность цветов элемента**

```
.photo_saturate {
    filter: saturate(300%);
}
```

**sepia() — придаёт элементу эффект сепии**

```
.photo_sepia {
    filter: sepia(80%);
}
```



**комбинирование фильтров — позволяет применять несколько эффектов одновременно**

```
.photo_multiple {
    filter: contrast(155%) brightness(130%);
}
```

# background-blend-mode

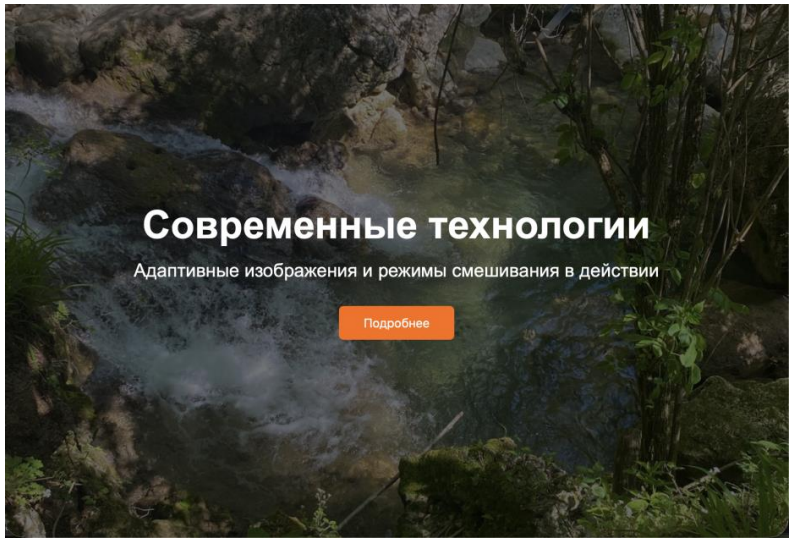
- Свойство background-blend-mode задаёт способ смешивания **нескольких фоновых слоёв** элемента (цвета и изображений).
- Работает **только внутри background одного элемента**.
- **Где применяется?**

Применяется к любым блочным элементам с несколькими фонами:

- div
- section
- header
- hero-блоки

Требует минимум **2 фоновых слоя**!

```
/* ✅ background + blend */
background:
  linear-gradient(rgba(0,0,0,0.55), rgba(0,0,0,0.55)),
  url("../resource/images/hero-desktop.jpg") center/cover no-repeat;

background-blend-mode: multiply;
```



# mix-blend-mode

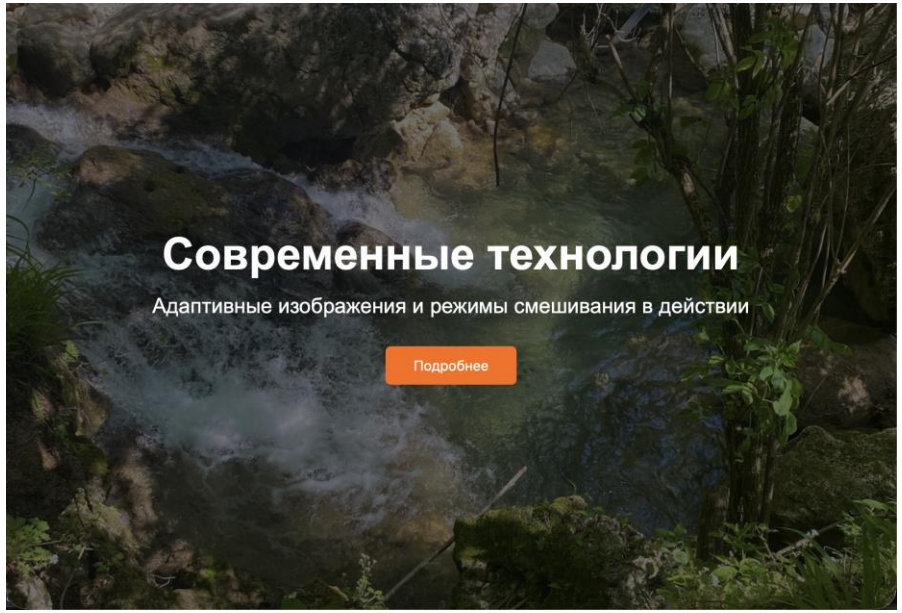
- mix-blend-mode задаёт, **как элемент смешивается с фоном родителя**.
- В отличие от background-blend-mode, он работает **между элементами**, а не внутри одного background.

```
/* ✅ mix-blend-mode */
mix-blend-mode: screen;

/* для читаемости */
text-shadow: 0 2px 10px rgba(0,0,0,0.6);
```

**Где применяется?**

- Можно применять к:
- тексту
- изображениям
- блокам
- иконкам

Работает относительно **родительского фон**



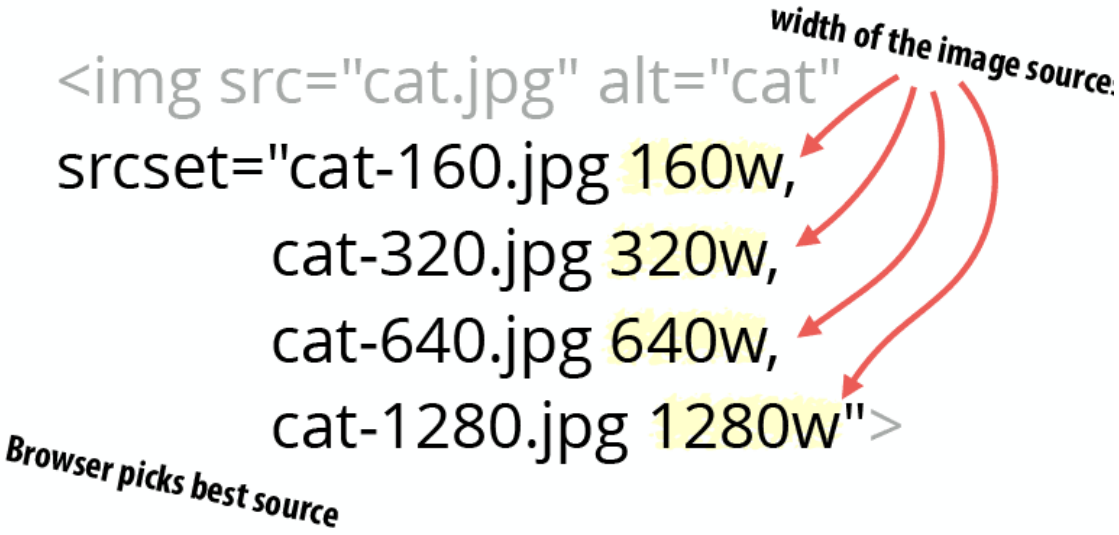
# Мета-тег viewport

- viewport управляет **масштабом и шириной страницы на мобильных устройствах**.
- Без него сайт на телефоне отображается как десктопный.

```
  <!-- ✅ viewport -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Hero Blend Demo</title>
  <link rel="stylesheet" href="css/2_hero.css">
</head>
<body>
```

# srcset (адаптивные изображения)

- srcset позволяет браузеру выбрать **наиболее подходящее изображение** под устройство.
- Экономит трафик
  Ускоряет загрузку
  Улучшает адаптив



# Picture

- <picture> позволяет:
- менять изображения по условиям
- использовать разные форматы
- делать art-direction
- Более мощный, чем просто img.

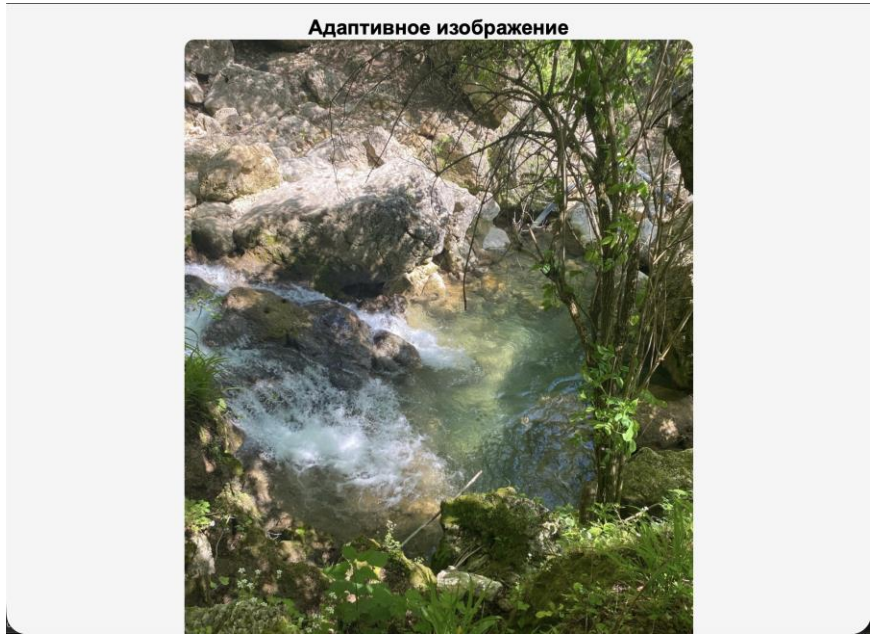
```
<!-- ✅ picture + srcset -->
<picture>
  <source
    media="(max-width: 600px)"
    srcset="resource/images/hero-mobile.jpg">

  <source
    media="(max-width: 1024px)"
    srcset="resource/images/hero-tablet.jpg">

  <img
    src="resource/images/hero-desktop.jpg"
    alt="Пример адаптивного изображения"
    loading="lazy"
    class="demo-image__img">
</picture>
</section>
```

# loading="lazy"

- Атрибут loading="lazy" включает **ленивую загрузку изображений**.
- Картинка грузится только когда приближается к экрану

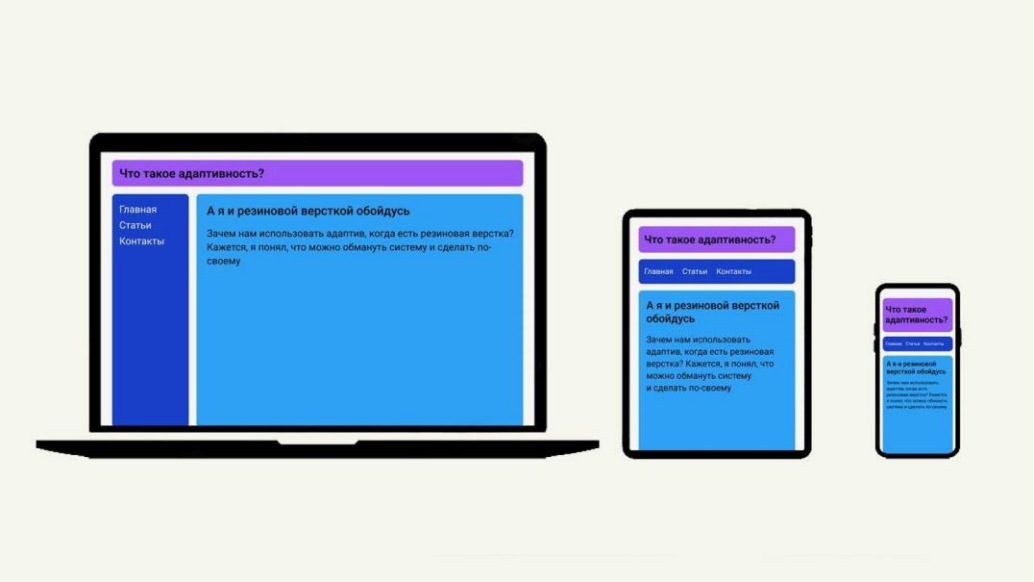
- улучшает:
- скорость страницы
- Core Web Vitals
- UX

```
<img
  src="resource/images/hero-desktop.jpg"
  alt="Пример адаптивного изображения"
  loading="lazy"
  class="demo-image__img">
```



# Адаптивная верстка

- Адаптивная верстка — это подход к разработке интерфейсов, при котором сайт корректно отображается на разных устройствах: телефонах, планшетах, ноутбуках и больших мониторах.
- Главная идея — не создавать отдельные версии сайта, а сделать один гибкий интерфейс, который подстраивается под размер экрана пользователя.
- Почему это важно:
- - более 60% трафика — мобильные устройства;
- - улучшение пользовательского опыта;
- - повышение доступности;
- - положительное влияние на SEO.



# Media Queries

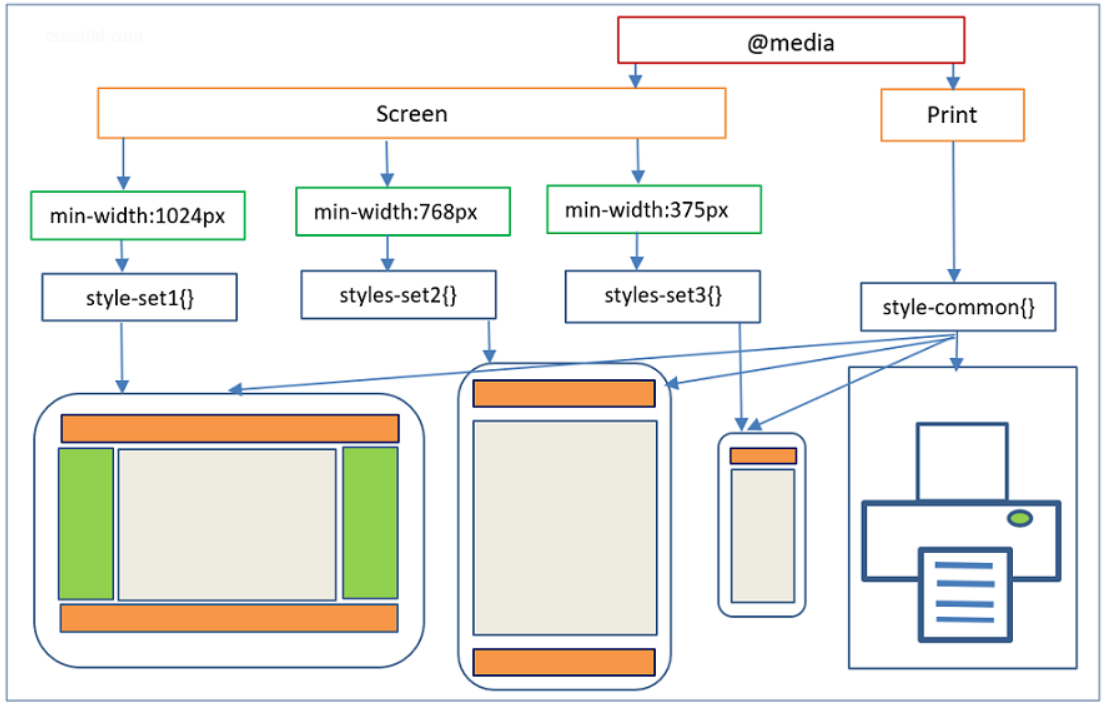
- **Что это такое?**
- Media Queries — это механизм CSS, который позволяет применять стили в зависимости от характеристик устройства или окна браузера.
- Проще говоря, мы задаём условие, и если оно выполняется — применяются определённые стили.

```
@media (max-width: 768px) {
  body {
    background: lightblue;
  }
}
```

- Это означает: если ширина экрана меньше или равна 768px — фон станет голубым.

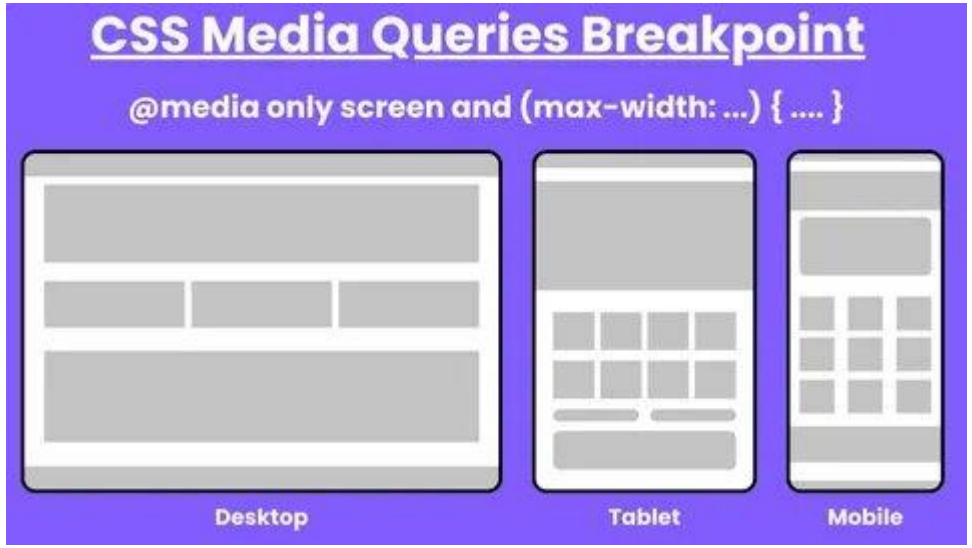
# Media Queries

- **Какие условия можно использовать:**
- width / height
- min-width / max-width
- orientation (portrait / landscape)
- resolution
- **Где применяются:**
- изменение расположения блоков
- изменение размера текста
- скрытие или показ элементов
- адаптация меню
- Media Queries — это основной инструмент адаптивной верстки.



# Breakpoints (точки перелома)

- **Что это?**
- Breakpoint — это ширина экрана, при которой интерфейс меняет структуру.
- Например:
- - при 1200px — 3 колонки
- - при 768px — 2 колонки
- - при 480px — 1 колонка




- Важно понимать, что точки перелома выбираются не **“по устройствам”**, а по тому моменту, когда дизайн начинает выглядеть плохо.

# Подходы к адаптивной верстке

- **Mobile-first**

Сначала пишутся стили для мобильных устройств, затем через min-width добавляются стили для больших экранов.

```
.container {
  width: 100%;
}

@media (min-width: 768px) {
  .container {
    width: 750px;
  }
}
```

**Преимущества:** - быстрее загружается на мобильных - проще масштабировать - современный стандарт - код становится чище

- **Desktop-first**

Сначала создаётся версия для больших экранов, затем через max-width происходит упрощение интерфейса.

```
.container {
  width: 1200px;
}

@media (max-width: 768px) {
  .container {
    width: 100%;
  }
}
```

**Минус** — сложнее поддерживать, часто получается больше переопределений.

# Гибкие единицы измерения

Адаптивность невозможна без относительных единиц измерения.

**%:**

```
width: 50%;
```

Процент вычисляется относительно родительского элемента. Используется для ширины блоков и сеток.

**em:**
Зависит от font-size родителя. Если родитель 16px, то 2em = 32px.
Особенность — значения могут накапливаться (вложенность влияет на размер).

**rem:**

```
margin: 2rem auto;
padding: 1.5rem;
```

Зависит только от размера шрифта в теге html.
Если html = 16px: - 1rem = 16px - 2rem = 32px
Преимущество — предсказуемость и удобство масштабирования.

**vw и vh:**
1vw = 1% ширины окна
1vh = 1% высоты окна
Пр

```
height: 100vh;
```

Элемент займёт всю высоту экрана.

**vmin и vmax:**
vmin — меньшее значение между vw и vh
vmax — большее значение
Используются реже, но полезны для адаптивной типографики.

# Гибкие единицы измерения

**Сравнение единиц:**

**em vs rem:**

- em зависит от родителя. rem зависит от html.
- В крупных проектах чаще используют rem, чтобы избежать накопления размеров.

**px vs rem:**

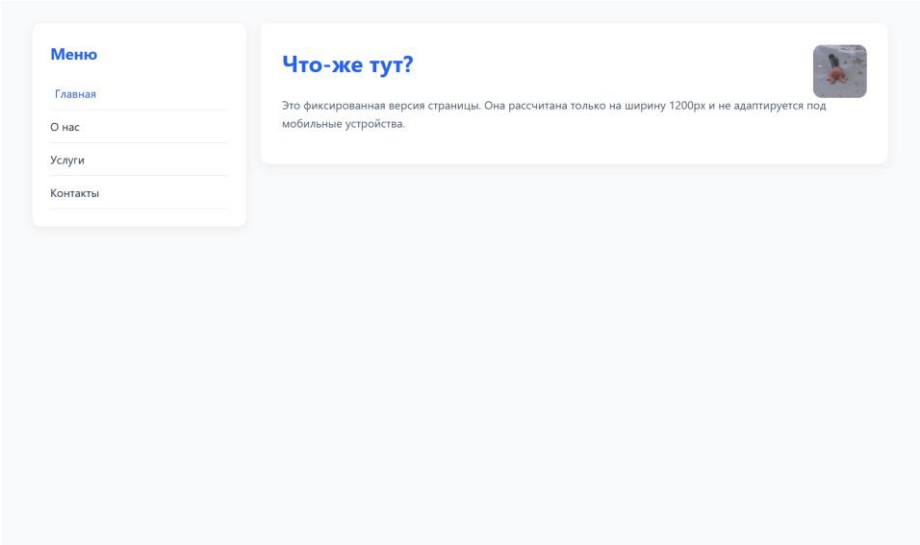
- px — фиксированная единица. rem — масштабируемая.
- Для доступности и адаптивности лучше использовать rem.

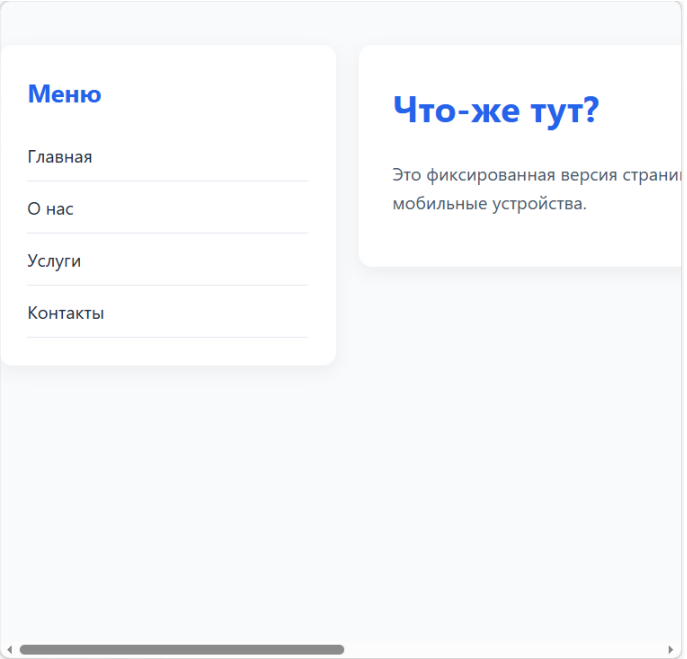
**vh vs %:**

- vh зависит от окна браузера. % зависит от родителя.
- Для полноэкранных блоков удобнее vh.

# Разница между фиксированной и адаптивной версткой

**Фиксированная верстка:**

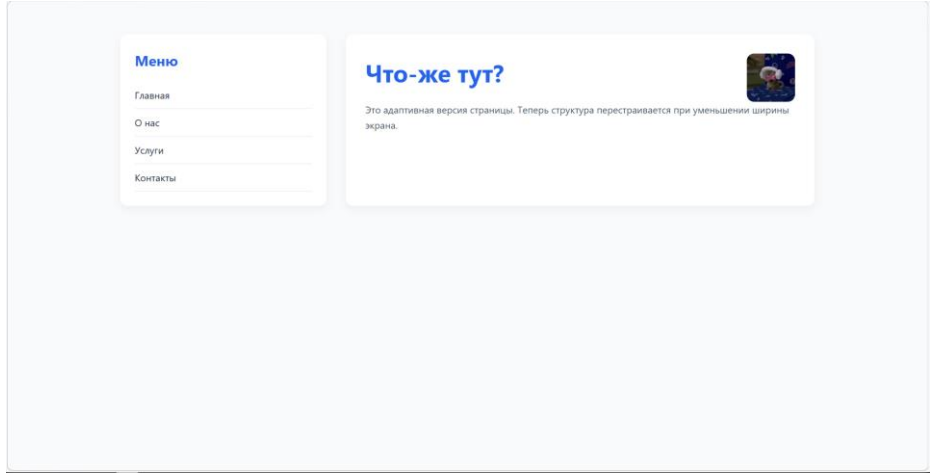


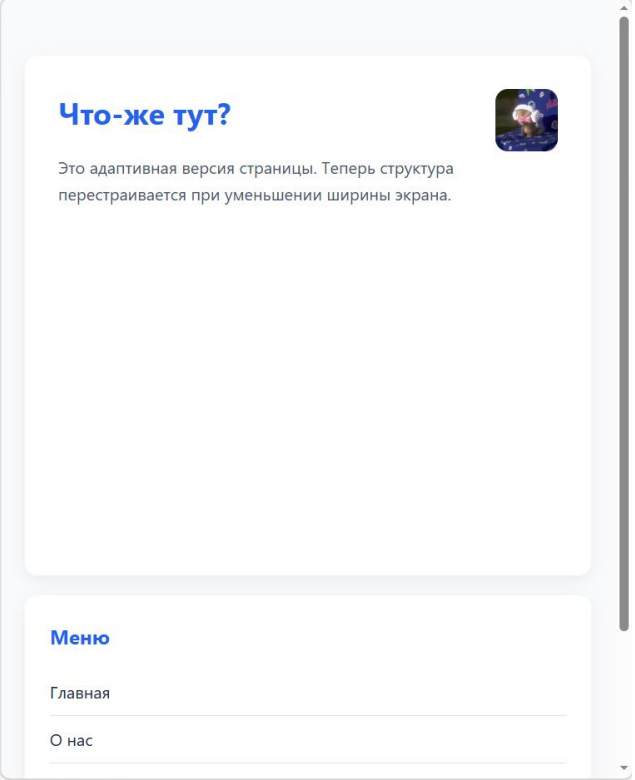




В фиксированной версии всё видно, но при уменьшении разрешения часть окна просто обрезается, так как она предназначена для ширины 1200px и более

# Разница между фиксированной и адаптивной версткой

**Адаптивная верстка:**




В адаптивной версии при изменении ширины страница перестраивается в другой формат